PA 0 - C++ Review - Analyzing Menu Files

Big idea: warm up your C++ by building a small class that reads a Menu Planner file and reports a few facts about it — a review of pointers, dynamic memory, classes, the rule of five, and a simple iterator.

Today

- what you build: `MenuModel` and `PairingFactor`
- the report your program prints
- managing memory by hand: raw arrays + the rule of five
- how it's graded on Gradescope

Recap — The Menu-Planner Files

menu3.menu

```
MENU

3
3 2 2

3
1 0
2 0 1
2 1 2

3
5 3 2

6
4 1 1 4 3 3

4
3 1 1 3
```

Four sections (blank-line separated)

- **1 — the `MENU` tag** (skip it)
- **2 — courses:** 3 courses with `3 2 2` dishes
- **3 — scopes:** for each table, which courses it covers (`1 0`, `2 0 1`, `2 1 2`)
- **4 — scores:** for each table, a count then its scores (`3/5 3 2`, . . .)

menu3.chosen

`1 1 1` — one course chosen in advance: course 1 (Drink) is dish 1.

Full story: the overview deck, *The Menu Planner*. This is the PA 0 reminder.

What You'll Build — All in `MenuModel.hpp`

Four types

- `struct Course` — two `int`s: a course's index (`courseIdx`) and its number of dishes (`numDishes`).
- `struct ChosenCourse` — two `int`s: which **course** (`courseIdx`) and which **dish**.
- `class PairingFactor` — one pairing table. It **owns** two raw arrays: `Course const** _scope` (**pointers** to the courses it covers) and `double* _scores` (the pairing scores).
- `class MenuModel` — the whole model. It **constructs and owns** the `Course`s, the `PairingFactor`s, and the chosen courses; each factor's scope **points into** its `Course` array. Parses the files; can be **iterated**.

Header-only — everything lives in `MenuModel.hpp` (the only file you submit). The provided `driver.cpp` (do not modify) builds a `MenuModel`, calls its methods, and prints the report. Because your classes own raw memory, each needs the full **rule of five**.

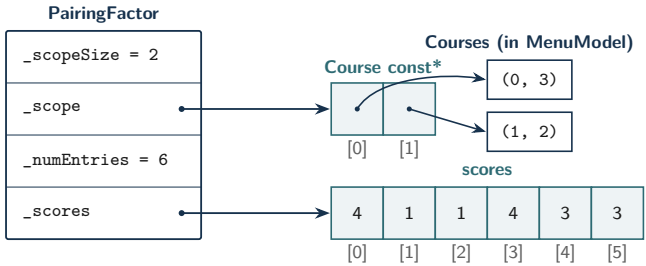
One PairingFactor Owns Two Arrays

Table 1: factor over (Main, Drink)

Main	Drink	score
Lentils	Cider	4
Lentils	Tea	1
Tofu	Cider	1
Tofu	Tea	4
Risotto	Cider	3
Risotto	Tea	3

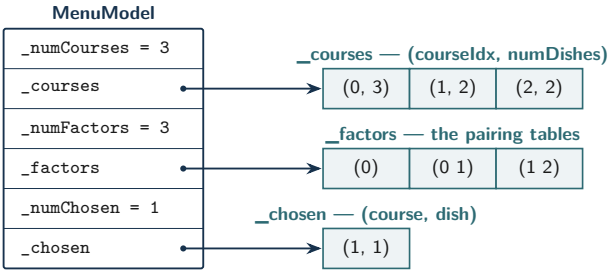
Read the **score** column down:

`4 1 1 4 3 3`



The factor's **score** column is the flat `_scores` array `4 1 1 4 3 3` this factor owns; `_scope` holds `Course const*` — **non-owning** pointers into the `MenuModel`'s `Courses`. The destructor frees the two arrays, never the `Courses`.

The MenuModel Owns Three Arrays



A `MenuModel` owns three arrays: the courses (`_courses` — each one a `Course` holding its `courseIdx` and `numDishes`), the pairing tables (`_factors` — each a `PairingFactor` whose scope just **points into** `_courses`), and the chosen courses (`_chosen`). `begin()`/`end()` walk `_factors`; like `PairingFactor` it allocates with `new[]`, frees in the destructor, and needs the full **rule of five**.

The Report Your Program Produces

Always (from the `.menu`)

```
courses:  3
dishes_per_course:  3 2 2
tables:  3
table_scopes:  (0) (0 1) (1 2)
table_sizes:  3 6 4
largest_table:  6
complete_meals:  12
```

Plus, if a `.chosen` is given

```
chosen_courses:  1
chosen:  (1 1)
free_courses:  2
```

`complete_meals` = the product of all dish counts ($3 \cdot 2 \cdot 2 = 12$); `table_sizes` = each table's number of scores; `free_courses` = courses − chosen. The autograder diffs your output against the reference **line for line**.

The C++ You'll Practice

Straight from Unit 0

pointers, `new`/`delete`, `nullptr`	0.1
references & `const`	0.2
classes & the **rule of five**	0.3
a simple **forward iterator**	0.5

(Templates, 0.4, come later — not in PA 0.)

Owning raw memory

`new[]` going in, `delete[]` coming out.

The rule of five via **copy-and-swap** is scaffolded in the starter — you fill in the **destructor** and the **deep-copy constructor**.

A copy of a `MenuModel` must own *its own* arrays — otherwise two objects free the same memory (a double-free). That's exactly what the rule of five prevents.

Rules & How It's Graded

Rules

- everything in `MenuModel.hpp`; don't modify `driver.cpp`
- standard library for **input only** (`string`, `ifstream`)
- raw `new[]`/`delete[]` arrays — **no** `std::vector` or `std::map`
- private members start with a leading `_` (e.g. `_scope`)
- no leaks, no double-frees (checked with sanitizers)

How it's graded

- the autograder **generates** many menu files (varied shapes/sizes, some chosen, some **re-randomized each run**)
- **memory first:** a leak / double-free **scores 0**
- then it **calls your methods** and checks each **return value** against the reference (not your printed output)
- every test is **fully shown** (model + per-method expected vs. yours); **no hidden tests**
- submit `MenuModel.hpp` only

Only the *generator* stays server-side — test against the provided `samples/` locally first.

Takeaways

- PA 0 is a **C++ warm-up**: read a Menu Planner file and report facts about it
- your classes **own raw arrays** — allocate with `new[]`, free in the destructor, and give each the full **rule of five**
- private data members use a **leading underscore** (`_scope`, `_scores`)
- the grader is fully transparent — you always see the menu, the expected output, and yours

Class Question

Your `PairingFactor` owns two arrays. Which *one* special member, if you leave it out, makes a copied `MenuModel` crash with a double-free — and which idiom in the starter keeps that from happening?